

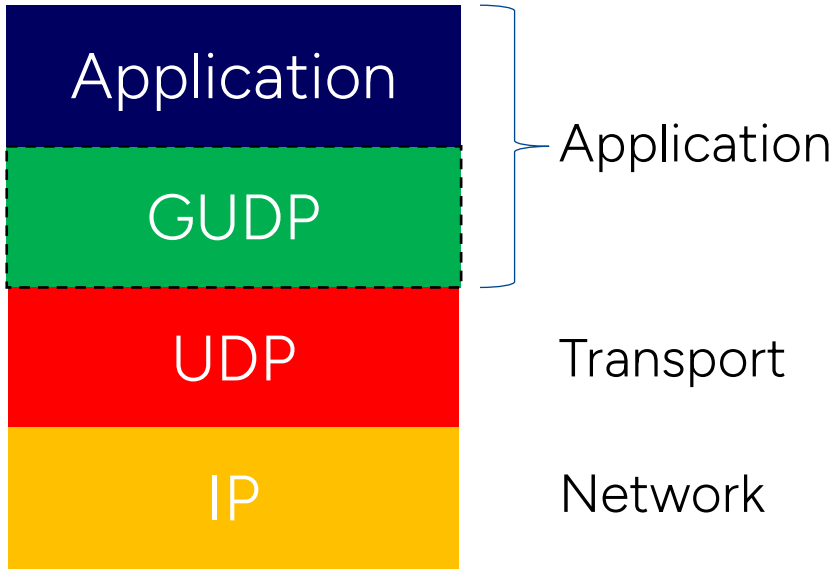
IK2215 Programming Assignment Introduction

Voravit Tanyingyong

Programming assignment overview

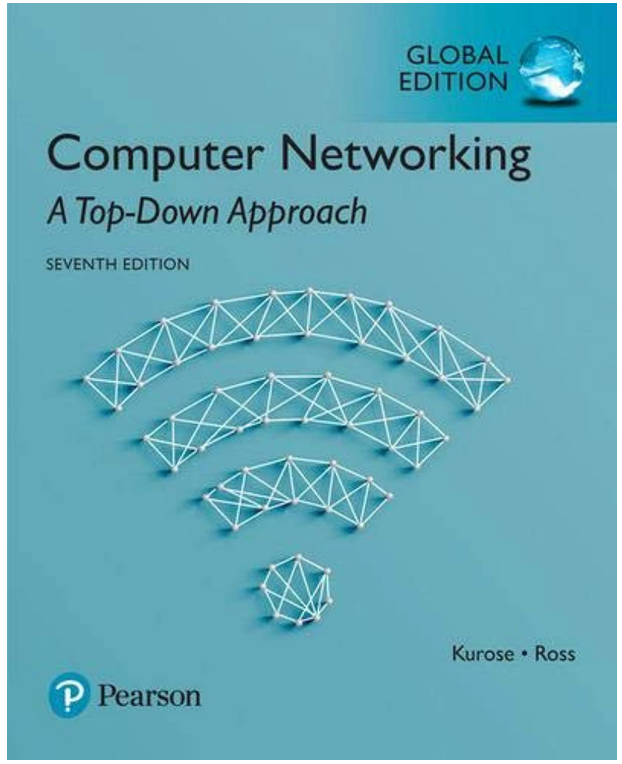
- Design and implement a reliable protocol for sending/receiving datagrams

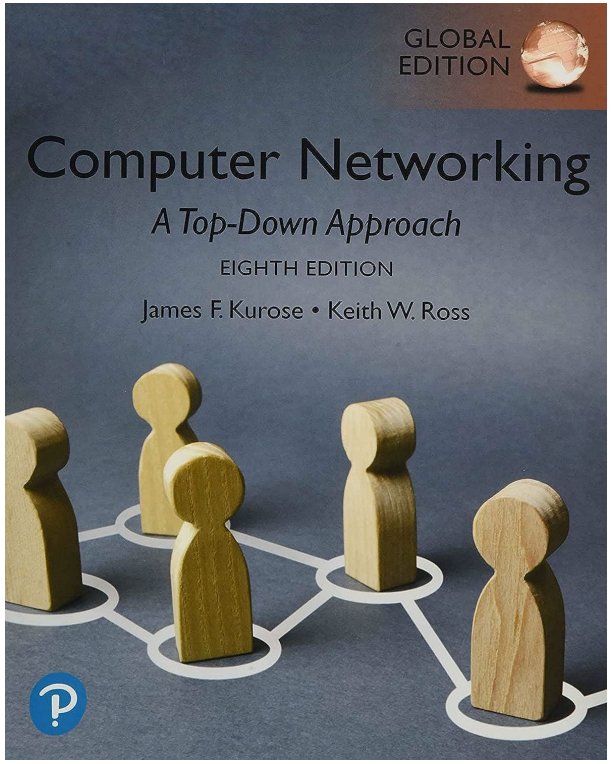
- Guaranteed UDP (GUDP)
  - Enabling reliable transport over UDP
    - Based on the Go-Back-N (GBN) protocol
  - Automatic repeat request (ARQ)
    - Use acknowledgements and timeouts for reliable transmission
  - Sliding window flow control
    - Multiple packets in flight
  - Asynchronous communication
    - Unlike TCP, GUDP is not connection-oriented (no connection establishment)



Reliable data transfer and Go-Back-N

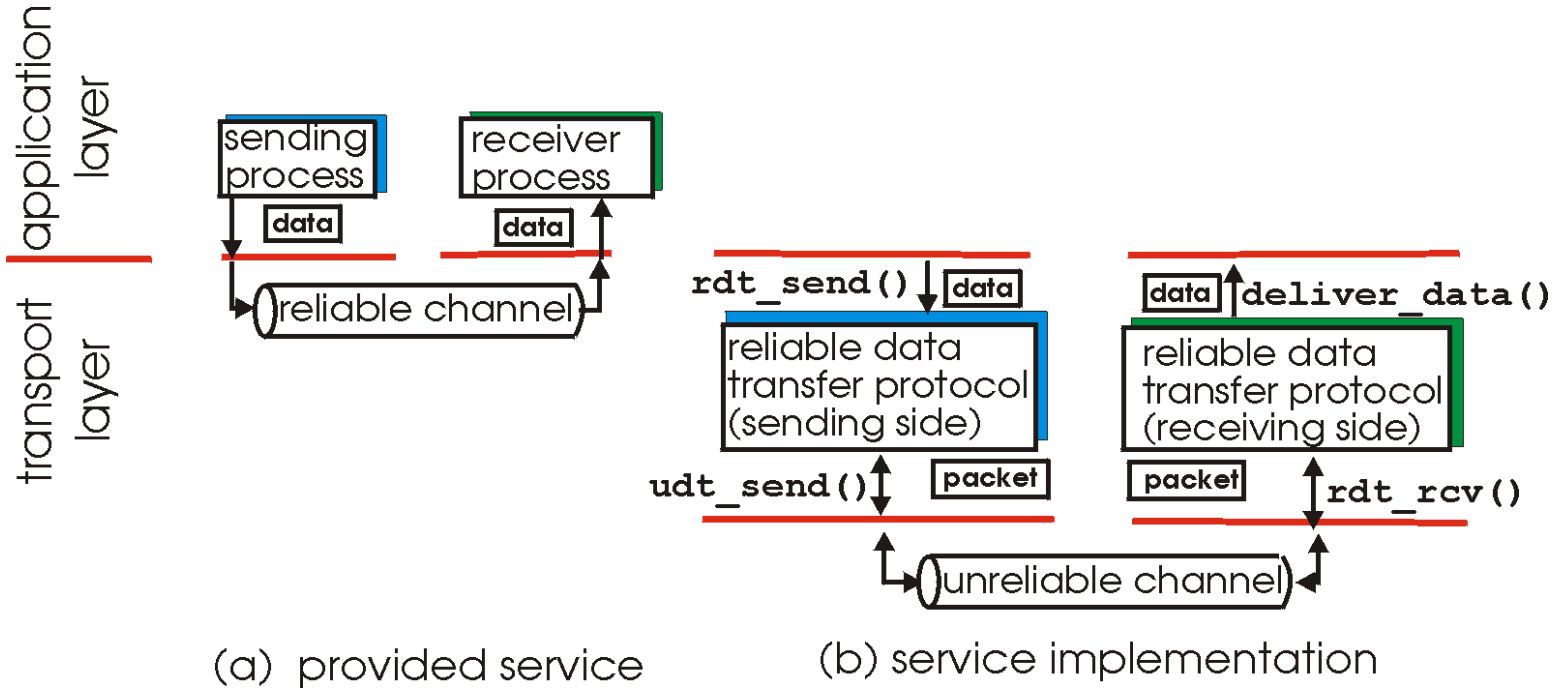
- Computer Networking: a Top-Down Approach
  J.F. Kurose, K.W. Ross
- Chapter 3.4 – 3.4.3
- Slides from the authors
  - Slides 4 – 8 and 12 – 14





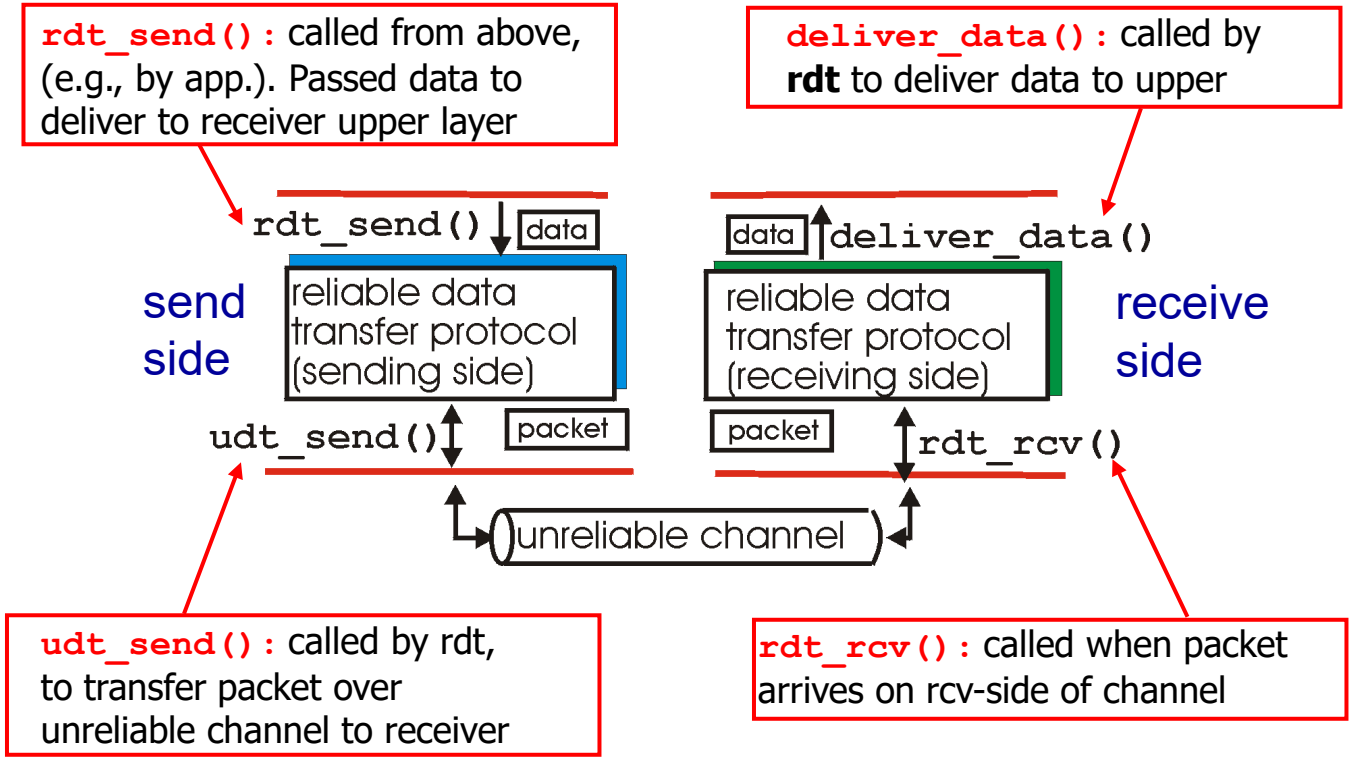
Principles of reliable data transfer

- One of the most important challenges in networking



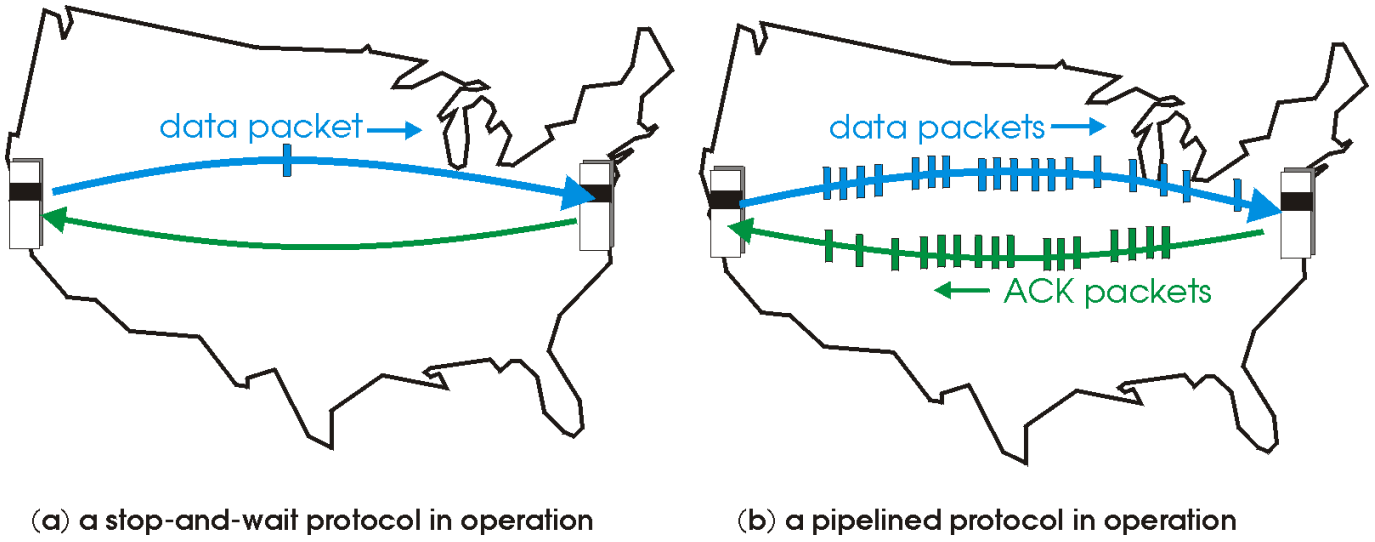
- Characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

Reliable data transfer: interfaces



Pipelined protocols

- Pipelining: sender allows multiple, “in-flight”, yet-to-be-acknowledged pkts
  - Range of sequence numbers must be increased
  - Buffering at sender and/or receiver

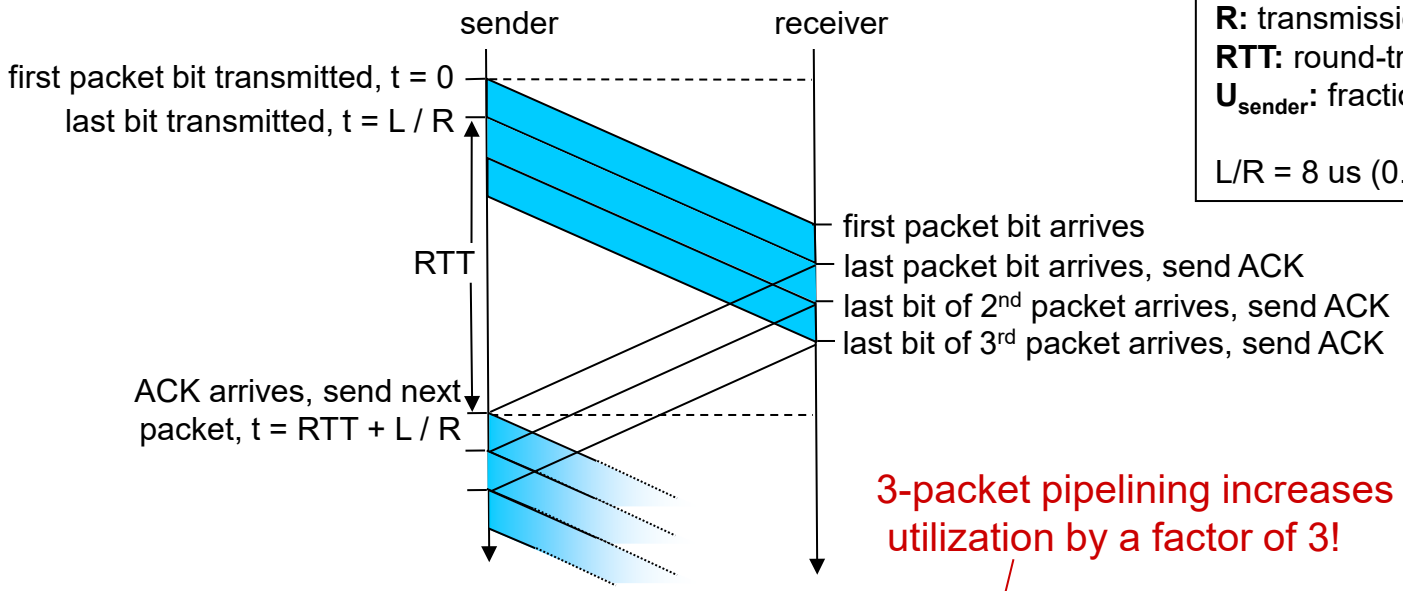


(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

- Two generic forms of pipelined protocols: Go-Back-N, selective repeat

Pipelining: increased utilization



L: a packet size (8000 bits for 1000 bytes)
R: transmission rate (10^9 bps for 1 Gbps)
RTT: round-trip-time (~ 30 ms for speed-of-light)
U_{sender}: fraction of time the sender is busy sending bits into the channel

L/R = 8 us (0.008 ms)

3-packet pipelining increases utilization by a factor of 3!

$$U_{sender} = \frac{3L / R}{RTT + L / R} = \frac{.0024}{30.008} = 0.00081$$

Pipelined protocols: overview

Go-Back-N (GBN)

- Sender can have up to N unack'ed packets in pipeline
- Receiver only sends cumulative ack
  - Doesn't ack packet if there's a gap
- Sender has timer for oldest unacked packet
  - When timer expires, retransmit *all* unacked packets

Selective Repeat

- Sender can have up to N unack'ed packets in pipeline
- Receiver sends individual ack for each packet
- Sender maintains timer for each unacked packet
  - When timer expires, retransmit only that unacked packet

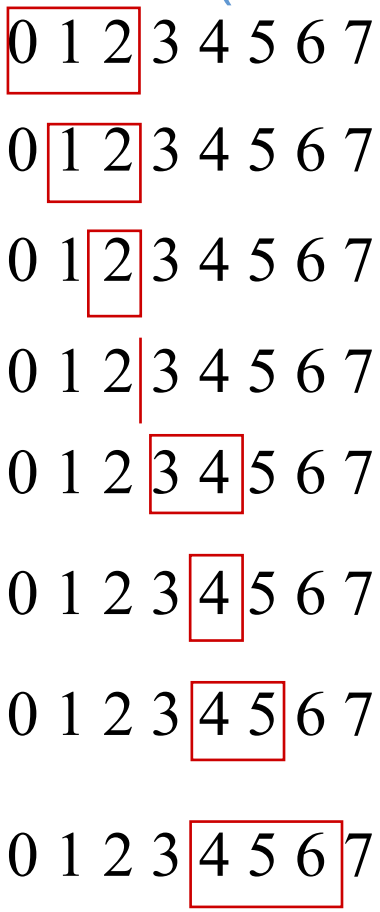
GBN sliding window protocol

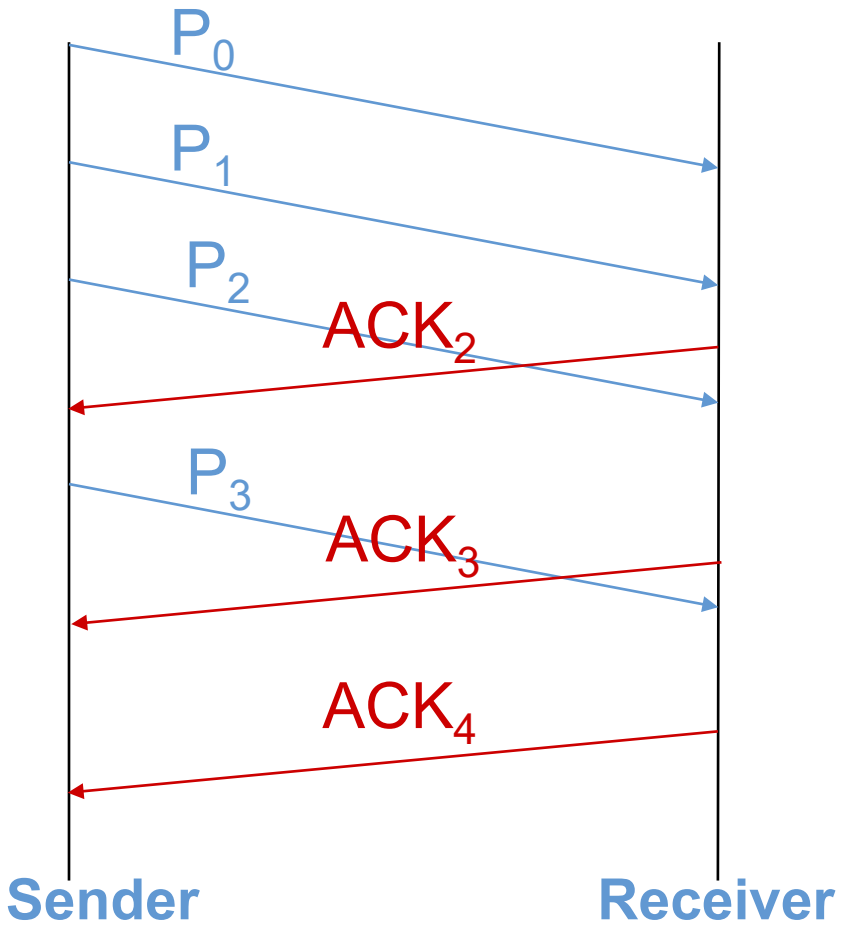
- Understand the details of a basic sliding window protocol
- An ACK is an ACK (and not a NACK)
  - The receiver sends an ACK only if it receives the next packet in sequence*
  - You cannot use an ACK to tell the sender that a packet has been lost (i.e., no NACK)
  - No duplicate ACK detection
- The sender increases the window in accordance with the ACK
- Retransmissions are triggered by timeouts (and nothing else)
  - Receiving an ACK with unexpected sequence number does not trigger a retransmission

* There can be a problem in this case. We will this dicuss it later on.

Sliding window flow control

Window (size 3)





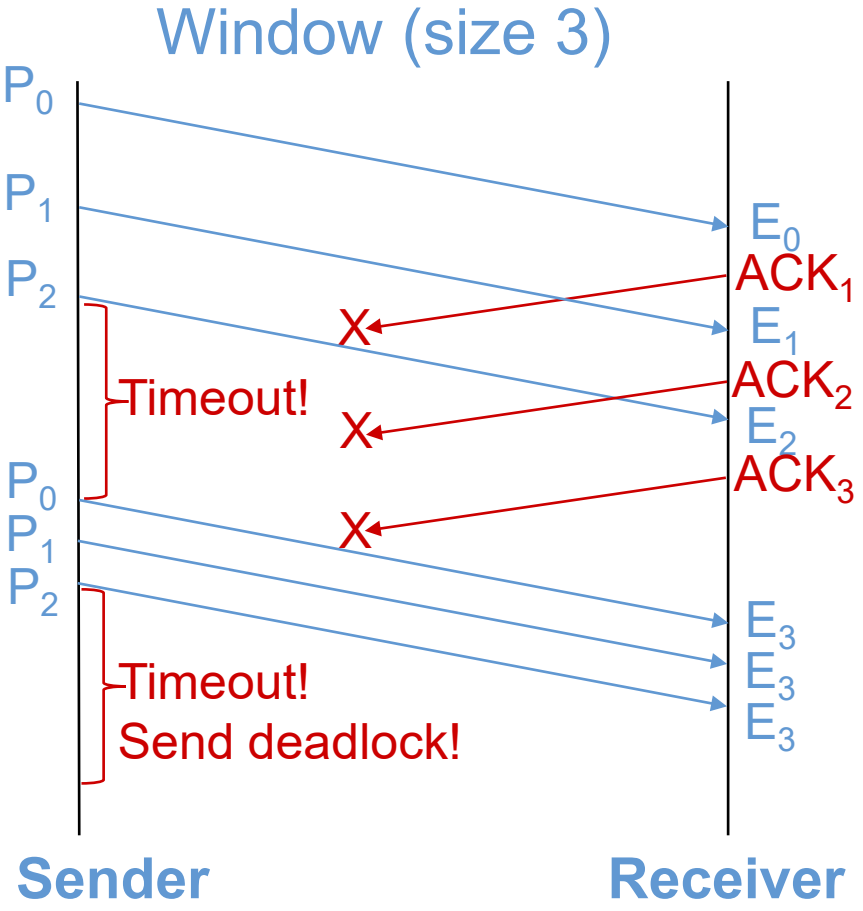
Problem when receiver only ACKs the next sequence

Problem:

- If the receiver sends an ACK only if it receives the next packet in sequence, a deadlock occurs when all ACKs (= number of window size) were lost

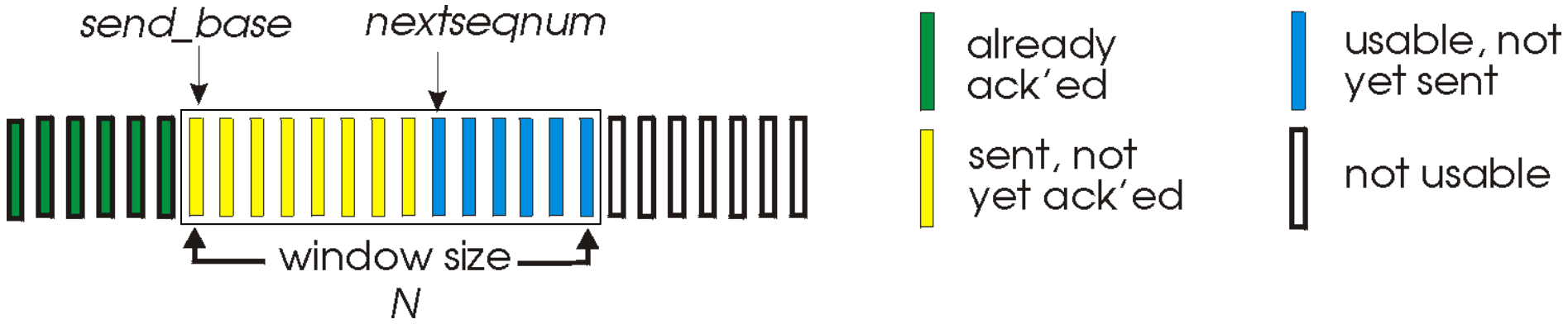
Solution:

- Receiver must send ACK with the expected sequence number when it receives a packet with a different sequence number than the expected sequence number
- Sender upon receiving an ACK can assume all packets with (ACK sequence number - 1) were received successfully



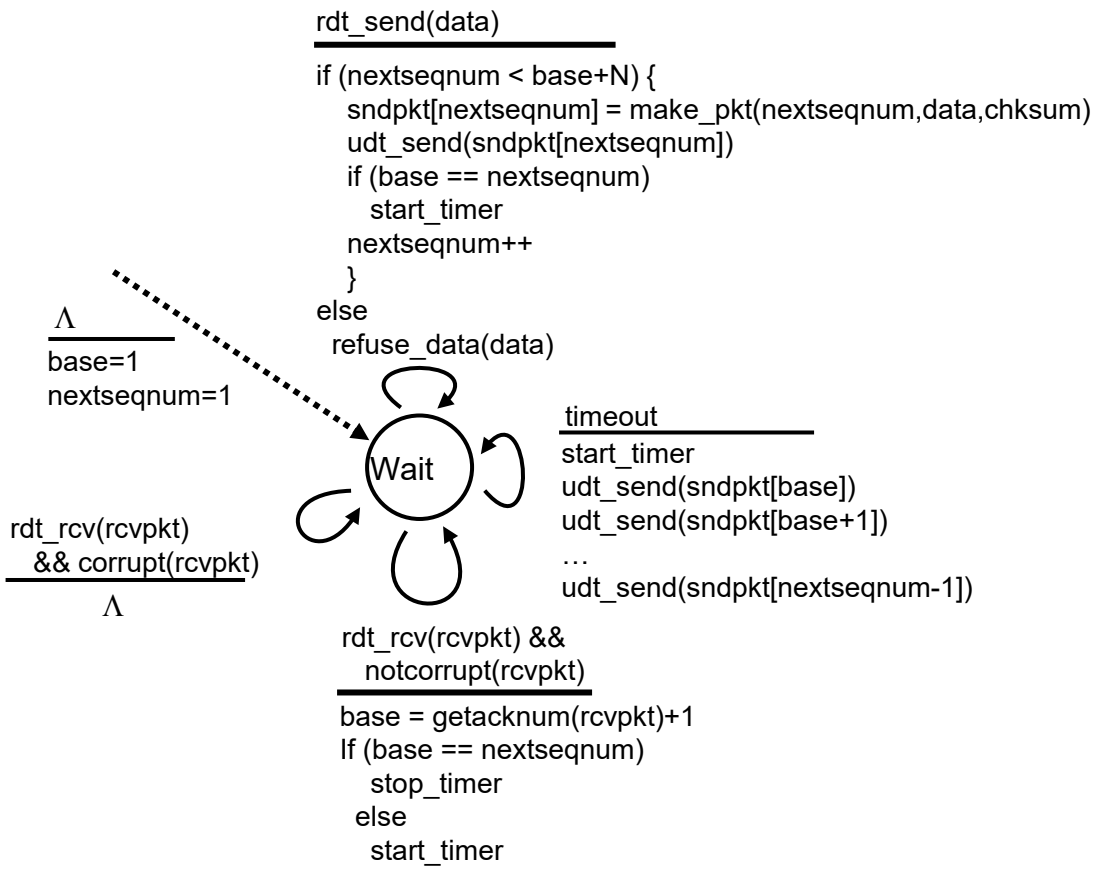
Go-Back-N sender

- k-bit seq # in pkt header (range of sequence numbers is [0, 2^k - 1])
- "window" of up to N, consecutive unack'ed pkts allowed



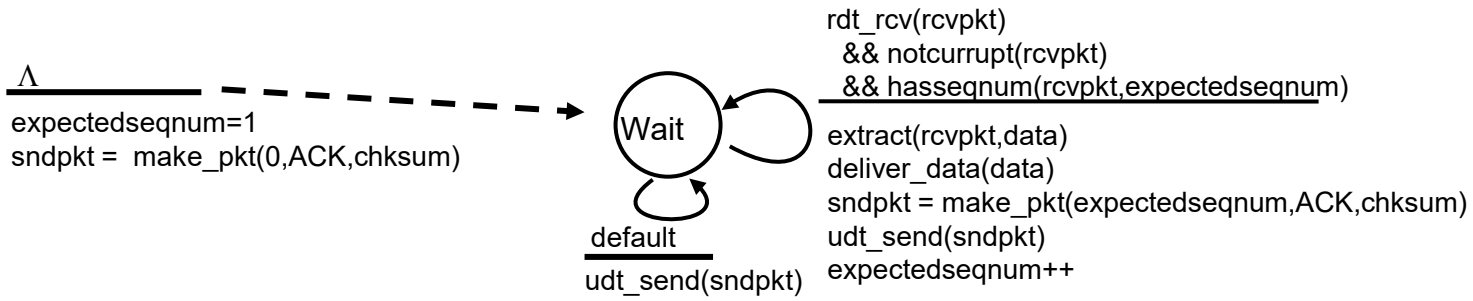
- ACK(n): ACKs all pkts up to, including seq # *n* - "cumulative ACK"
  - On receiving ACK(n): move window forward to begin at *n+1*
- Timer for oldest in-flight pkt
- Timeout(n): retransmit packet n and all higher seq # pkts in window
- TCP implementation sends the next expected sequence in the ACK, i.e., ACK(*n*) ack'ed all pkts up to *n-1*
  - GUDP implementation will also send the next expected sequence in the ACK

GBN sender extended FSM*



* See the course book chapter 3.4.3, figure 3.20

GBN receiver extended FSM*



- ACK-only: always send ACK for correctly-received pkt with highest *in-order* seq #
  - May generate duplicate ACKs
  - Need only remember **`expectedseqnum`**
- out-of-order pkt:
  - Discard (don't buffer): *no receiver buffering!*
  - Re-ACK pkt with highest in-order seq #

* See the course book chapter 3.4.3, figure 3.21

GUDP implementation in java

- GUDP runs in user space, in the same process as the application

We provide:

You are not allowed to modify these files!

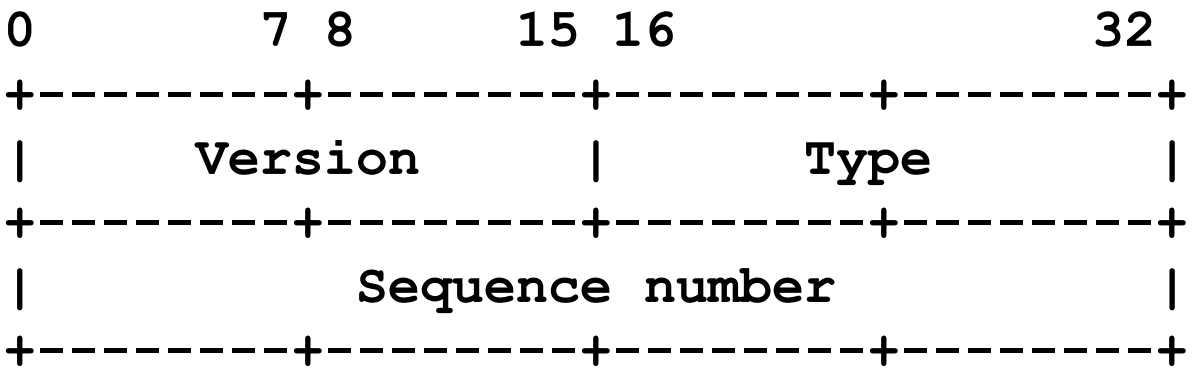
- GUDPPacket.java: A class for GUDP protocol declarations with associated methods to access the GUDP packet header and payload
- GUDPSocketAPI.java: Well-defined API that you must use for your implementation
- GUDPEndPoint.java: A class for keeping track of remote endpoints
- GUDPSocket.java: A class for GUDP library
- GUDPSender.java: A class that monitors send buffers and sends packets when they are in the buffer.
- GUDPReceiver.java: A class that receives packets from remote endpoints and puts them in the buffers.



- ARQ
- Sliding window flow control

GUDP header

- Version: version of the GUDP protocol
  - Use version 1!
- Type: packet type
  - DATA (1), BSN (2), ACK (3), and FIN (4)
- How to use sequence numbers:
  - DATA packets: increases by one for each packet sent
  - BSN packets: random
  - ACK packets: sequence number of next expected DATA packet
  - FIN packets: sequence number of last DATA packet plus one



GUDPSocketAPI.java – API you must use

```
import java.net.DatagramPacket;
import java.io.IOException;

public interface GUDPSocketAPI {

    public void send(DatagramPacket packet) throws IOException;
    public void receive(DatagramPacket packet) throws IOException;
    public void finish() throws IOException;
    public void close() throws IOException;
}
```

- Your code must conform to this application programming interface (API)
- Class/method declarations defined for the assignment
- You will write the GUDPSocket class that implements this API
  - You may add variables, methods, and inner classes in GUDPSocket.java

GUDPSocket.java – skeleton code for GUDP

```
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.io.IOException;
public class GUDPSocket implements GUDPSocketAPI {
    DatagramSocket datagramSocket;
    public GUDPSocket(DatagramSocket socket) {
        datagramSocket = socket;
    }
    public void send(DatagramPacket packet) throws IOException {}
    public void receive(DatagramPacket packet) throws IOException {}
    public void finish() throws IOException {}
    public void close() throws IOException {}
}
```

- The skeleton above is incomplete
- The actual file contains more variables and descriptions of what must be done in each method

send()

```
public void send(DatagramPacket packet) throws IOException;
```

- Send a packet
- The application put data in the DatagramPacket format
- The destination address/port included in the DatagramPacket
- Non-blocking – returns immediately
  - Put application data in GUDP send buffer for delivery asynchronously
  - The DatagramPacket must be encapsulate in GUDP format before putting in the send buffer

receive()

```
public void receive(DatagramPacket packet) throws IOException;
```

- Receive a packet
- The application fetch the data from GUDP receive buffer, otherwise wait for the data to arrive
  - GUDP receives incoming packets from remote endpoints independently from the application
  - GUDP receive buffer stores packets in GUDP format, which must be decapsulated before delivering it to the application
- The application handles packets from different senders (which can be differentiated based on the information in the packet)

finish()

```
public void finish() throws IOException;
```

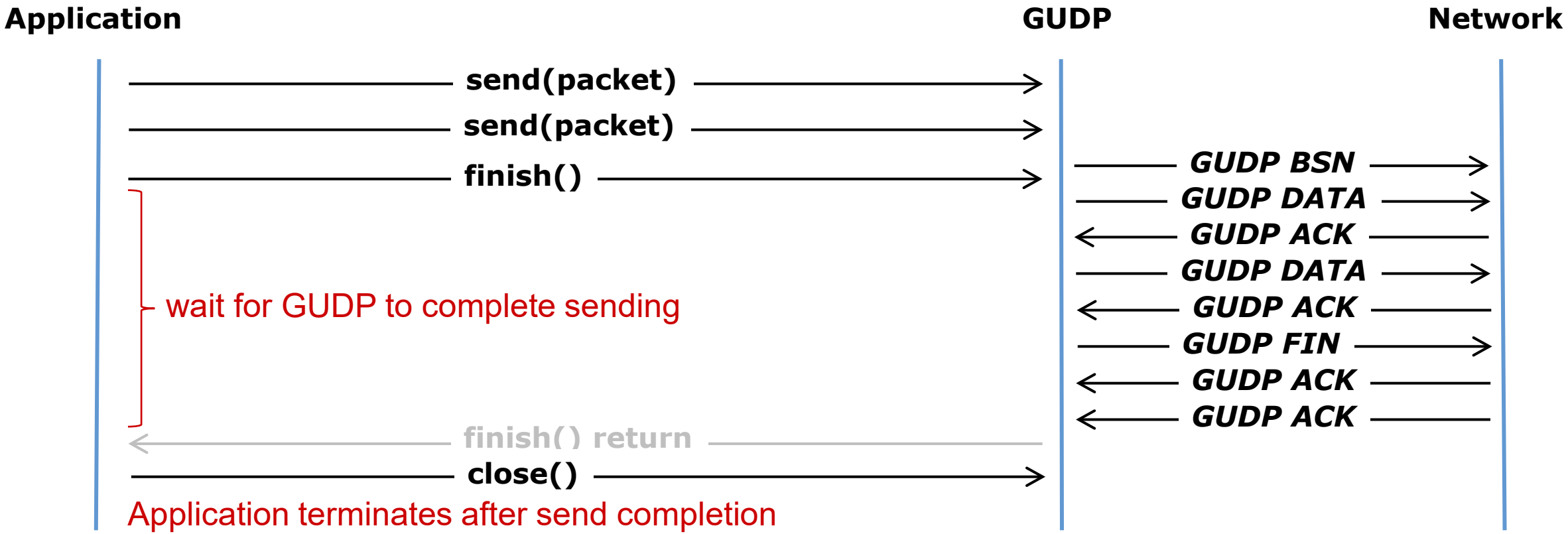
- Finish sending
- The application calls this method to inform GUDP that it's done sending
  - Add FIN to the GUDP send buffer for all remote endpoints to indicate that the transmission is completed
- Wait for GUDP to complete the actual sending and return when it is done, otherwise report error/timeout by throwing the IOException
  - GUDP may retransmit packets due to packet lost or arriving out-of-order
  - GUDP cleans up data structure that is used to track remote endpoints when transmission is completed

close()

```
public void close() throws IOException;
```

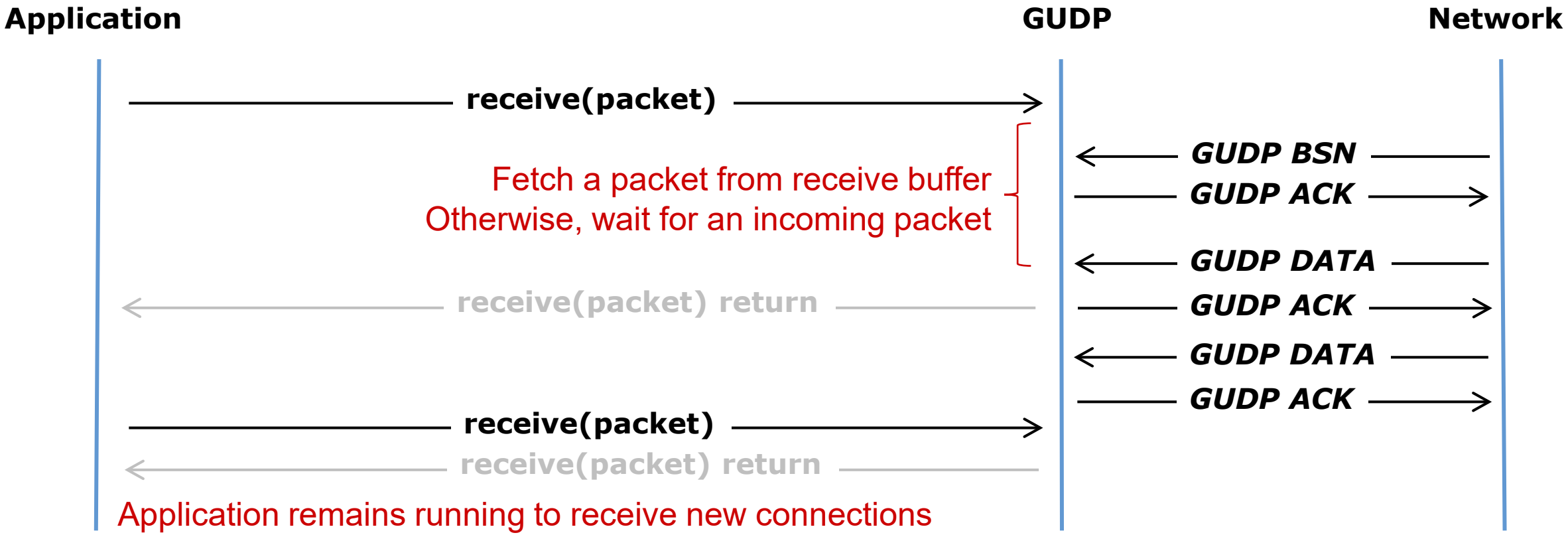
- The application calls this method to terminate the GUDP socket
- GUDP cleans up, stop sender and receiver threads, closes the socket, and return.

GUDP sender side



- Data transfer may happen after the application passed all packets to GUDP
- GUDP can send multiple packets (<= window size) before it receives any ACK

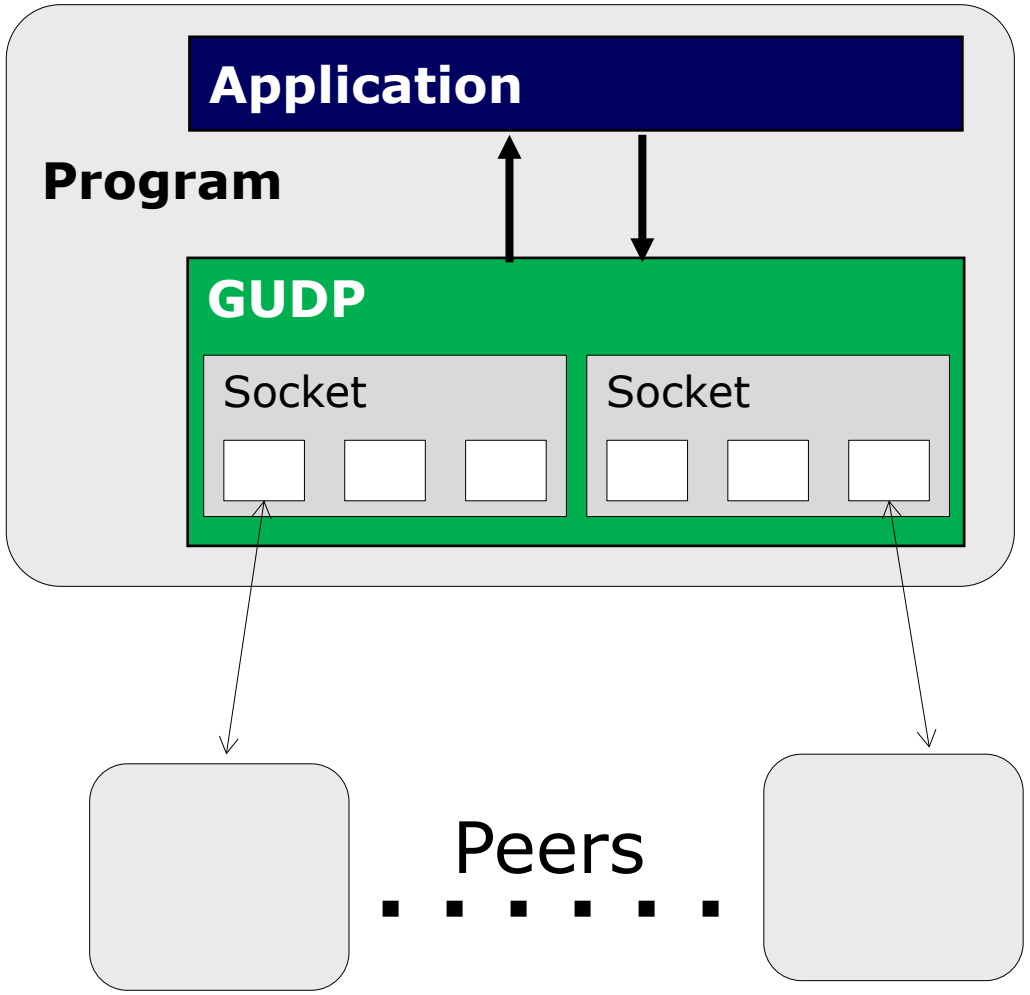
GUDP receiver side



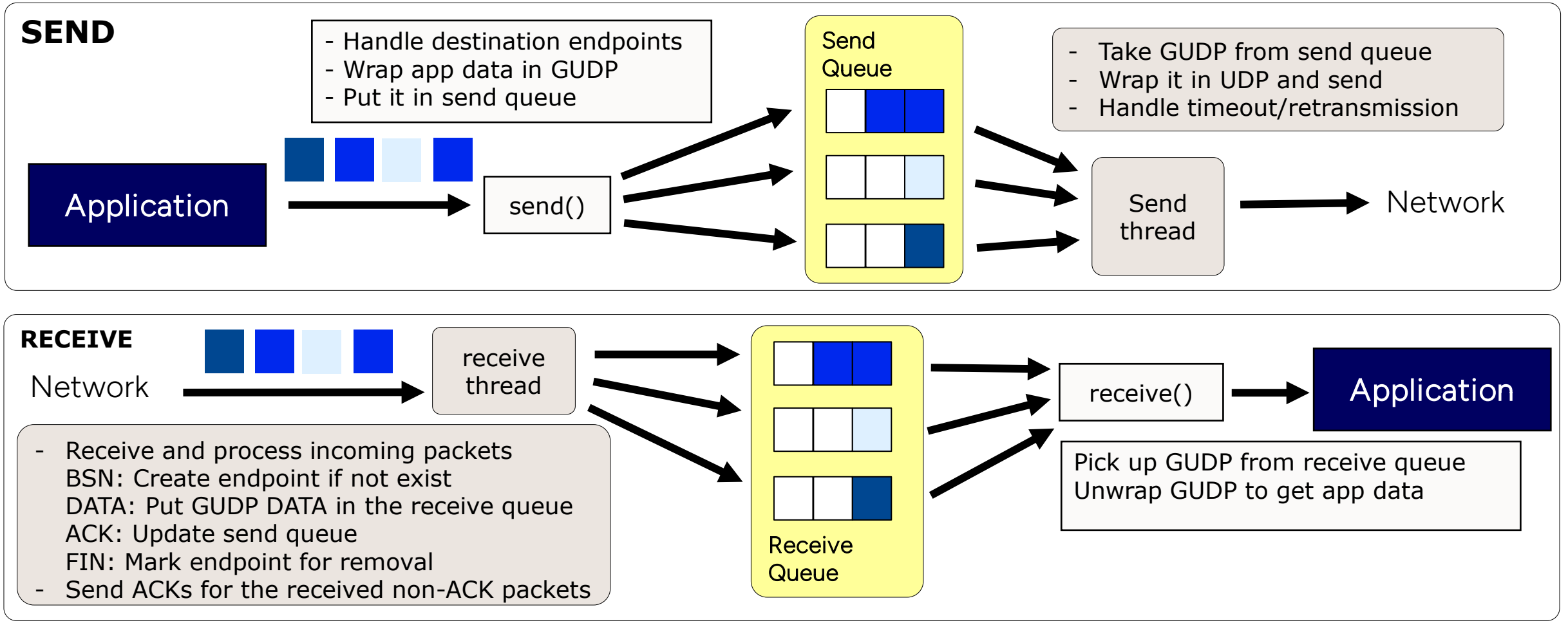
- Receive returns only after GUDP has DATA
- Receiver keeps the socket open to receive more DATA

Protocol control block

- An application can open multiple GUDP sockets
- Each GUDP socket can be used for communication with multiple peers
- Two levels
  - Multiple GUDP sockets
  - Multiple peers per socket
- Need to
  - Maintain state for per-socket “peers”
  - Have a way to look up peer state
  - Maintain queues with outbound/inbound packets



GUDP Implementation: Send and receive processes



Main tasks

Part 1: GUDPSocket.java

- Implement core functionalities, including send(), receive(), finish(), and close() methods
- Assume GUDPSender.java and GUDPReceiver.java are already implemented as described in the comments in the files

Part 2: GUDPSender.java

- Implement the send thread that monitors send queues and sends packets based on the GBN protocol when there are packets in the queues
- Assume GUDPSocket.java and GUDPReceiver.java are implemented as described in the comments in the files
  - GUDPSocket.java from the teacher, not from your part 1 submission!
  - GUDPReceiver.java handles ACK for the GBN sender, and it also calls FSMSender after it removes all ACKed packet from the send buffer

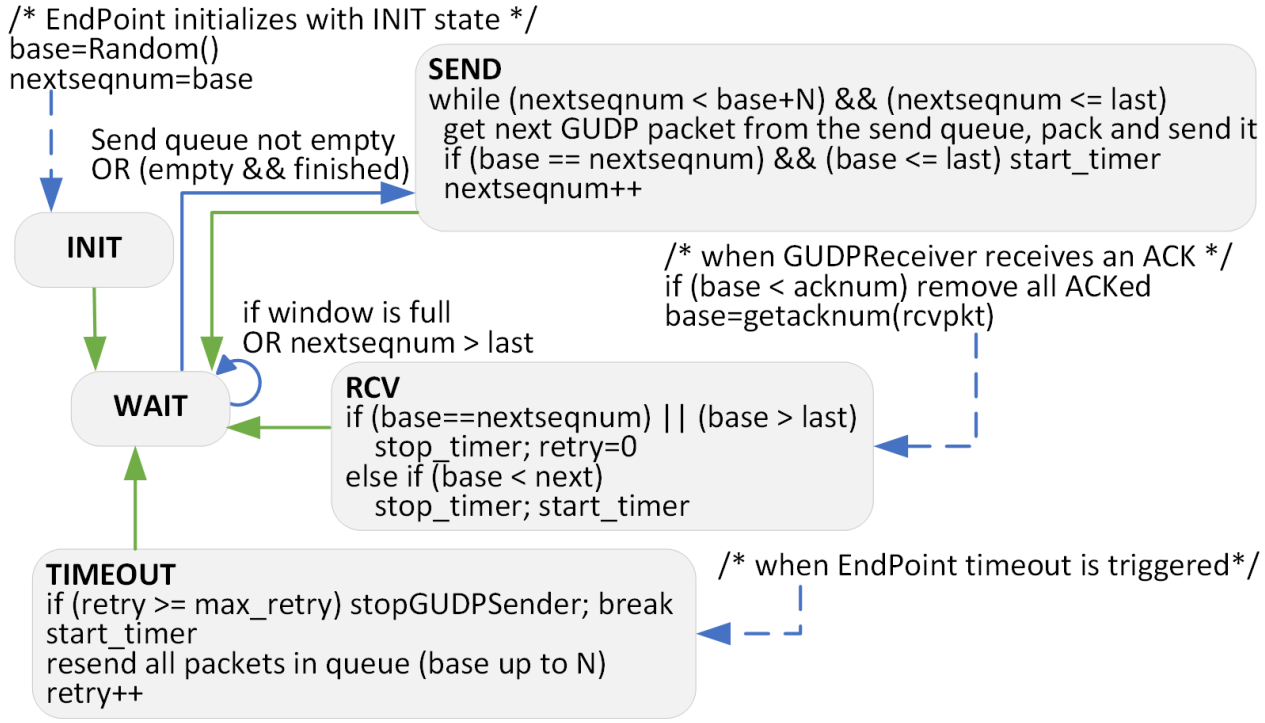
Teacher implementation: GUDPSender class

```
while (runFlag)
    for each endpoint in senderQueue  /* implemented with iterator and while loop */
        run FSMSender(endPoint)
        if endpoint is finished and its send queue is empty, remove it from senderQueue

/* if GUDPSender is terminated before completing the transmission,
  * remove all endpoints so that the application can terminate gracefully
  */
if senderQueue is not empty, iterate through senderQueue to remove all endpoints
```

Teacher implementation: FSMSender method

```
switch (endPoint.state)
   case INIT: change endpoint.state to WAIT
   case WAIT:
      if (endpoint send queue is not empty) OR (if send queue is empty AND finished), change endpoint.state to SEND
      if endpoint send window is full OR nextseqnum > last
         put the thread to sleep for 100 ms
   case SEND:
      while send window is not full AND nextseqnum <= last
         send a packet
         if base = nextseqnum AND base <= last, start timer
         nextseqnum++
      change endpoint.state to WAIT
   case RCV:
      if base = nextseqnum OR base > last, stop timer; retry=0
      else if base < nextseqnum, restart timer
      change endpoint.state to WAIT
   case TIMEOUT:
      if retry >= max_retry, stop GUDPSender
      start timer
      resend all packets that have not been ACKed
      retry++; change endpoint.state to WAIT
```



Teacher implementation: GUDPReceiver class

```
while (runFlag)
    When packet arrives, unpack it to GUDP format
    switch (packet.type)
    case ACK:
        remove all ACK'ed packets from senderQueue, update base
        move to RCV state and call FSMSender
    case BSN:
        if new end point, add a new GUDPEndPoint, update all parameters, add it to receiverQueue, and send ACK
        else if end point was finished
            reset end point parameters and use the new expectedseqnum based on BSN (seq+1), and send ACK
        else send ACK with the expectedseqnum
    case DATA:
        if seq==expectedseqnum, add GUDP packet to end point receive buffer, update all parameters, and send ACK
        else send ACK with the expectedseqnum
    case FIN:
        if seq==expectedseqnum, update all parameters, set end point as finished, and send ACK
        else send ACK with the expectedseqnum
    put thread to sleep for 50 ms  /* to add delay between the processing of each receiving packet */
if receiverQueue is not empty, iterate through receiverQueue to remove all endpoints
if senderQueue is not empty, iterate through senderQueue to remove all endpoints
```

Grading overview

The application should be able to:

- Send one or multiple files to one or more destinations
- Receive one or multiple files from one or more sources
- Handle unexpected situations gracefully
- Work with other implementations

- To pass, your submission must ensure that:
- The application can at least send and receive one file on one destination correctly
  - GUDP must be used in data transmission (show on the wire correctly)
  - Sliding window flow control is working correctly (multiple packets in-flight)
  - ARQ mechanism is working correctly (handle packet loss correctly)
- Your scores meet the grading criteria (see details on the next two slides)

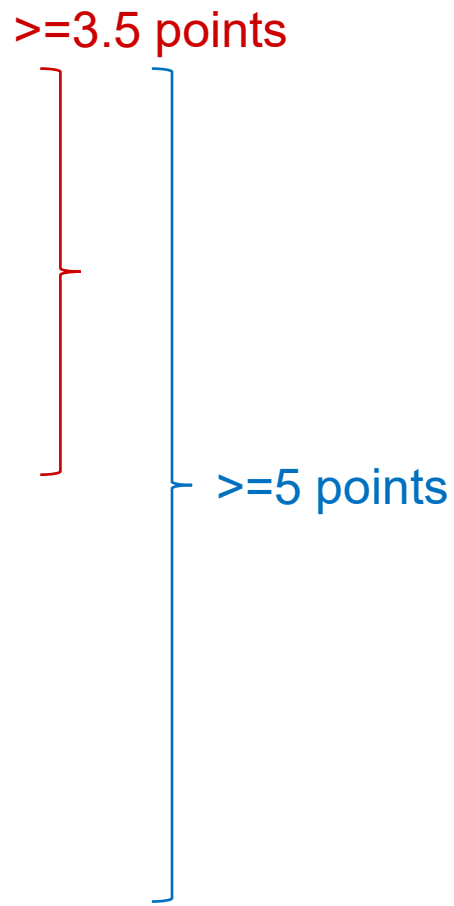
- Part 1: Deadline: Tue 16 Sep 17:00 sharp
- Part 2: Deadline: **Thu 25** Sep 17:00 sharp

- Make-up deadline: Fri 19 Sep 17:00 sharp
- Make-up deadline: **Tue 30** Sep 17:00 sharp

Part 1: Grading criteria

Score at least 3.5 points from the bold items and 5 out of 7 in total

1. **Send multiple packets in-flight + check packet content** (1)
2. **Send and receive files with your code without loss** (1)
3. **Send one file to other receiver without loss** (0.5)
4. **Send one file to other receiver with loss** (0.5)
5. **Receive one file from other sender without loss** (0.5)
6. **Receive one file from other sender with loss** (0.5)
7. Send one file to multiple receivers without loss (0.5)
8. Send one file to multiple receivers with loss (0.5)
9. Send multiple files to other receiver without loss (0.5)
10. Send multiple files to other receiver with loss (0.5)
11. Receive multiple files from other sender without loss (0.5)
12. Receive multiple files from other sender with loss (0.5)



Part 2: Grading criteria

Score at least 2.5 points from the bold items and 3.5 out of 5 in total

1. **Send multiple packets in-flight + check packet content** (1)
2. **Send and receive files with your code without loss** (1)
3. **Send one file to other receiver without loss** (0.5)
4. **Send one file to other receiver with loss** (0.5)
5. ~~**Receive one file from other sender without loss**~~ ~~(0.5)~~
6. ~~**Receive one file from other sender with loss**~~ ~~(0.5)~~
7. Send one file to multiple receivers without loss (0.5)
8. Send one file to multiple receivers with loss (0.5)
9. Send multiple files to other receiver without loss (0.5)
10. Send multiple files to other receiver with loss (0.5)
11. ~~Receive multiple files from other sender without loss~~ ~~(0.5)~~
12. ~~Receive multiple files from other sender with loss~~ ~~(0.5)~~

>=2.5 points

>=3.5 points

Plagiarism*

- Plagiarism in practical work and computing code
- “It is important that students ‘do their own work’ when they write computer code, when document an experiment, create a design or answer a mathematical problem. If they do not do these activities themselves, yet claim the results as their own, this is plagiarism.”

- Students who, with unauthorized aids or otherwise attempt to mislead the exam or when a student's performance is otherwise to be assessed, may lead to disciplinary action.

- You may discuss but **do not share your code with other classmates**!

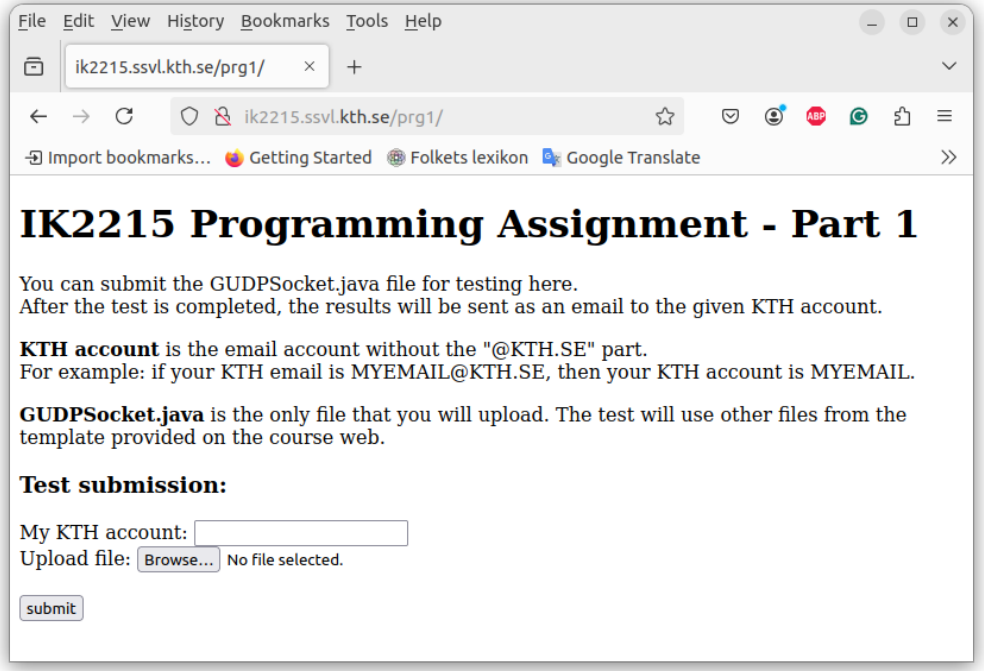
* More information on KTH webpage about Cheating and plagiarism

Testing

- We provide sample applications that you can use to run with your GUDP code
  - VSFtp.java: A class for a simple file transfer protocol
  - VSSend.java: An application for sending files over VSFtp
  - VSRecv.java: An application for receiving files over VSFtp
- You are responsible for identifying relevant test cases and performing tests
  - Need to complete GUDPSocket.java, GUDPSender.java, and GUDPReceiver.java
- Think through the protocol carefully and know how it should work exactly
- Think through the dynamic behaviour of the GUDP library
  - What happens, and when?
- Define the protocol states and transitions
  - <current state, event, action, new state>
- If you have question:
  - Discussion forum: Q&A for lab activities
  - Q&A sessions for verbal discussion or additional support

Test service on http://ik2215.ssvl.kth.se

- Only accessible within KTH network
  - From outside KTH network, you must connect via KTH VPN-service
- Part 1: http://ik2215.ssvl.kth.se/prg1
- Part 2: http://ik2215.ssvl.kth.se/prg2
- You must provide:
  - Your KTH account i.e., KTH email without the “@KTH.SE” part
  - Your submission file
    - Part 1: GUDPSocket.java
    - Part 2: GUDPSender.java
- The test runs at 11:00 and 23:00 everyday
  - Slow: 5-10 minutes per submission
- Results send to the KTH account you provided



Example test output for Part 1

```
OK: Code compiles without error.
### TEST1: Send multiple packets in-flight + check packet content (1.0p)
OK: GUDP version must be 1
OK: First packet is GUDP BSN (type 2)
OK: Sequence number is random and not zero or one
OK: BSN packet contains only GUDP header
OK: GUDP version must be 1
OK: Second packet is GUDP DATA (type 1)
OK: Sequence number should be random and not zero
OK: Second packet has an increment sequence number
OK: data packet seems to contain GUDP header + payload
TEST1: OK       1.0p
### TEST2: send and receive files with your code without loss (1.0p)
OK: Your code can send and receive one file
OK: Your code can send and receive multiple files
TEST2: OK       1.0p
### TEST3: send one file to other receiver without loss (0.5p)
OK: Your code can send one file to other receiver
TEST3: OK       0.5p
### TEST4: send one file to other receiver with loss (0.5p)
OK: Your code can send one file when first BSN is lost
OK: Your code can send one file when first DATA is lost
OK: Your code can send one file when first FIN is lost
OK: Your code can send one file when first ACK is lost
OK: Your code can send one file with random loss
TEST4: OK       0.5p
### TEST5: receive one file from other sender without loss (0.5p)
OK: Your code can receive one file from other sender
TEST5: OK       0.5p
```

```
### TEST6: receive one file from other sender with loss (0.5p)
OK: Your code can receive one file when first BSN is lost
OK: Your code can receive one file when first DATA is lost
OK: Your code can receive one file when first FIN is lost
OK: Your code can receive one file when first ACK is lost
OK: Your code can receive one file with random loss
TEST6: OK       0.5p
### TEST7: send one file to multiple receivers without loss (0.5p)
OK: Your code can send one file to multiple receivers
TEST7: OK       0.5p
### TEST8: send one file to multiple receivers with loss (0.5p)
OK: Your code can send one file to multiple receivers
TEST8: OK       0.5p
### TEST9: send multiple files to other receiver without loss (0.5p)
OK: Your code can send multiple files to other receiver
TEST9: OK       0.5p
### TEST10: send multiple files to other receiver with loss (0.5p)
OK: Your code can send multiple files to other receiver
TEST10: OK      0.5p
### TEST11: receive multiple files from other sender without loss (0.5p)
OK: Your code can receive one file from other sender
TEST11: OK      0.5p
### TEST12: receive multiple files from other sender with loss (0.5p)
OK: Your code can receive one file from other sender
TEST12: OK      0.5p
##########
IMPORTANT: You pass only if scores of TEST1-6 >=3.5 points and TEST1-12 >=5.0 points.
You get the scores only when you pass. Otherwise, you get 0 points
RESULTS: PASS
SCORE: 7.0
##########
```

Example test output for Part 2

```
OK: Code compiles without error.
### TEST1: Send multiple packets in-flight + check packet content (1.0p)
OK: GUDP version must be 1
OK: First packet is GUDP BSN (type 2)
OK: Sequence number is random and not zero or one
OK: BSN packet contains only GUDP header
OK: GUDP version must be 1
OK: Second packet is GUDP DATA (type 1)
OK: Sequence number should be random and not zero
OK: Second packet has an increment sequence number
OK: data packet seems to contain GUDP header + payload
TEST1: OK       1.0p
### TEST2: send and receive files with your code without loss (1.0p)
OK: Your code can send and receive one file
OK: Your code can send and receive multiple files
TEST2: OK       1.0p
### TEST3: send one file to other receiver without loss (0.5p)
OK: Your code can send one file to other receiver
TEST3: OK       0.5p
### TEST4: send one file to other receiver with loss (0.5p)
OK: Your code can send one file when first BSN is lost
OK: Your code can send one file when first DATA is lost
OK: Your code can send one file when first FIN is lost
OK: Your code can send one file when first ACK is lost
OK: Your code can send one file with random loss
TEST4: OK       0.5p
TEST5: SKIPPED                  0.0p
TEST6: SKIPPED                  0.0p
```

```
### TEST7: send one file to multiple receivers without loss (0.5p)
OK: Your code can send one file to multiple receivers
TEST7: OK       0.5p
### TEST8: send one file to multiple receivers with loss (0.5p)
OK: Your code can send one file to multiple receivers
TEST8: OK       0.5p
### TEST9: send multiple files to other receiver without loss (0.5p)
OK: Your code can send multiple files to other receiver
TEST9: OK       0.5p
### TEST10: send multiple files to other receiver with loss (0.5p)
OK: Your code can send multiple files to other receiver
TEST10: OK      0.5p
TEST11: SKIPPED                 0.0p
TEST12: SKIPPED                 0.0p
##########
IMPORTANT: You pass only if scores of TEST1-6 >=2.5 points and TEST1-12 >=3.5 points.
You get the scores only when you pass. Otherwise, you get 0 points
RESULTS: PASS
SCORE: 5.0
##########
```

Caveat

- 15% drop probability. However, in practice, it can be much higher
  - 9 retransmission attempts
- Iterators are weakly consistent for ConcurrentLinkedQueue and LinkedBlockingQueue
  - May run into inconsistent data
- Concurrency issues with multiple threads accessing GUDPEndpoint
  - GUDPSender and GUDPReceiver may start/stop timer concurrently, causing unexpected behavior
  - Timer task may be left running, preventing the program from terminating
- Teacher's code is not error-free
  - A thread may throw an error, causing the test case to fail and resulting in point deduction

Useful resources

- Course book: 8th and 7th edition
  - Read Chapter 3.4 through Chapter 3.4.3 Go-Back-N (GBN)
- TCP Operational Overview and the TCP Finite State Machine (FSM)
- Producer-consumer in Java: Baeldung, geeksforgeeks
- Java queue implementations: Oracle, Baeldung, geeksforgeeks,
- Java documentation for different classes:
  - DatagramSocket, DatagramPacket,
  - ConcurrentLinkedQueue, LinkedBlockingQueue

What you need to do

- Read through relevant materials thoroughly
  - Guaranteed UDP (GUDP) page on Canvas
  - This programming introduction slides
  - Read through the given source code template for the assignment
- Familiarize with java programming
- Plan early and work incrementally
- Submit your code to the test service daily!